

TypeCity Analytics

Conversation Summary · May 2026

Overview

This conversation developed a full analytics project — TypeCity — by pivoting an existing Lean 4 formal specification of a fictional city into a data analytics portfolio project. The project demonstrates SQL, pandas, Python, CSV data modeling, and Streamlit dashboarding skills, using the TypeCity hierarchy as the underlying domain.

The Superstore Tableau dashboard was used as the reference aesthetic: a rich, multi-metric explorer built on top of a clean dimensional model. TypeCity replicates that structure with its own synthetic urban dataset.

1. Background — The Lean 4 Specification

TypeCity was originally a Lean 4 project using a city metaphor to illustrate core concepts of dependent type theory (DTT). The canonical spec (TypeCity.lean) defines:

- A four-level geographic hierarchy: Quarter (8x8 mi), District (4x4 mi), Neighborhood (2x2 mi), Cell (1x1 mi) — $4 \times 4 \times 4 \times 4 = 256$ total cells.
- Two 9-stop subway lines: Orange (N-S) and Blue (E-W), crossing at Central Station (mile 8).
- Formally verified properties: subway line isomorphism, adjacency theorems, nested Sigma-type addresses.

The pivot to data analytics preserves the structure wholesale. The hierarchy becomes a dimensional model; the subway becomes a network analysis subject; the Sigma-type address (Quarter, District, Neighborhood, Cell) maps directly to referential integrity enforced by foreign keys — 'correct by construction' in DTT becomes a schema constraint in SQL.

2. Schema Design

Eight CSV files were designed before any data was generated. The schema is normalized enough to teach real SQL joins, but not over-engineered.

File	Rows	Key columns
quarters.csv	4	quarter_id, character, base_pop_density
districts.csv	16	district_id, quarter_id, name
neighborhoods.csv	64	neighborhood_id, district_id

cells.csv	256	cell_id, x, y, population, zoning_type, avg_household_income
subway_lines.csv	2	line_id, direction, color_hex
subway_stations.csv	18	station_id, name, x, y, is_transfer
station_lines.csv	18	station_id, line_id (junction table)
ridership.csv	3,024	station_id, day_of_week, hour, boardings, alightings

station_lines.csv is a deliberate teaching moment — joining through it to answer 'which stations are on the Orange line?' is your first real SQL JOIN.

3. Coordinate System

Cells occupy a 16x16 grid with integer coordinates $x = 0..15$ (W to E) and $y = 0..15$ (S to N). Quarter boundaries fall at $x = 7.5$ and $y = 7.5$:

```
NW: x 0-7, y 8-15 (Residential, high density)
NE: x 8-15, y 8-15 (Commercial, medium density)
SW: x 0-7, y 0-7 (Industrial, low density)
SE: x 8-15, y 0-7 (Mixed, medium density)
```

The Orange Line runs at $x = 8$ (N-S spine); the Blue Line runs at $y = 8$ (E-W spine). Central Station sits at (8, 8) — the geometric and transit center of the city. This geometry means the choropleth, subway overlay, and quarter borders all align on the same coordinate space.

4. Data Generation

generate_typecity.py produces all eight files from a seeded random generator (SEED = 42) for full reproducibility. Key design decisions:

- Quarter character drives synthetic data. NW (Residential) uses pop_mean = 4,200, income_mean = \$72k. NE (Commercial) uses pop_mean = 2,800, income_mean = \$95k. SW (Industrial) uses pop_mean = 1,400, income_mean = \$48k.
- Zoning weights per quarter: NW is 65% Residential; NE is 55% Commercial; SW is 55% Industrial; SE is 35% Mixed.
- Ridership uses a bimodal weekday profile (AM peak at 8am, PM peak at 5:30pm) and a flat midday weekend profile. Central Station has a 5x hub multiplier.

Sanity check results

```
Total population: 697,135
NW (Residential): 235,796 (33.8%)
SE (Mixed): 210,069 (30.1%)
NE (Commercial): 169,512 (24.3%)
SW (Industrial): 81,758 (11.7%)

Central Station weekly boardings (Blue side): 138,215
Central Station weekly boardings (Orange side): 119,114
```

Next busiest station: BL-4 West Inner at 34,089

Central Station's dominance (138k vs 34k for the next busiest stop) is the most visually striking feature of the ridership data.

5. Streamlit Choropleth Dashboard (app.py)

The first dashboard page is a metric choropleth of the 16x16 cell grid, built with Plotly inside Streamlit. Stack: Streamlit, Plotly, pandas. DuckDB was recommended as the SQL engine for future pages.

Choropleth mechanics

- cells.csv is pivoted to a 16x16 NumPy matrix with pivot(index='y', columns='x'). Rows are flipped (::-1) so y=15 (North) appears at the top.
- scaleanchor='x', scaleratio=1 enforces square cells so the city renders as a true square.
- xgap=1.5 / ygap=1.5 draws visible gutters between cells.
- A 16x16 hover-text matrix is precomputed once per metric change as HTML strings — keeps Plotly fast.

Sidebar controls

- Five metrics: Population, Avg Household Income, Residential Units, Commercial Units, Park Coverage %.
- Each metric has its own Plotly colorscale (Blues, Greens, Purples, Oranges, YlGn).
- Four independent overlay toggles: subway lines, stations, quarter borders, quarter labels.
- Quarter summary cards show population, share of city, and avg income — styled with per-quarter accent colors.

Below the map

- Stacked bar chart: zoning mix (%) per quarter.
- Collapsible raw data table with quarter and zoning filters.

6. Intended Learning Arc

The project is structured to teach the analytics stack in the natural order that complexity demands it:

Stage	Skill	What TypeCity teaches it through
1	CSV / data modeling	Designing the 8-file schema; understanding normalization and FK relationships
2	pandas	Loading, joining, groupby on the hierarchy; time-series resampling on ridership
3	SQL (DuckDB)	Window functions for station rankings; multi-table JOINS through station_lines
4	Streamlit	Dashboard layer — thin and clean when the data layer is solid

7. Identified Next Steps

Five concrete directions were flagged for future development:

- Ridership explorer page: hourly line charts, weekday vs. weekend comparison, station-level bar charts.
- SQL exercises page: wire in DuckDB, build canned queries (e.g. 'rank stations by boardings within each line' using window functions).
- Population vs. ridership correlation scatter: does cell-level population predict nearby station usage?
- Events / time-series table: add a synthetic permits or service-requests table to enable true trend analysis.
- Multi-page Streamlit app: break choropleth, ridership, SQL explorer, and raw data into separate pages.

8. Files Produced

The following files were generated during this conversation:

- generate_typecity.py — master data generation script (seeded, reproducible)
- typecity_data/quarters.csv (4 rows)
- typecity_data/districts.csv (16 rows, with real place names)
- typecity_data/neighborhoods.csv (64 rows)
- typecity_data/cells.csv (256 rows, 12 columns)
- typecity_data/subway_lines.csv (2 rows)
- typecity_data/subway_stations.csv (18 rows)
- typecity_data/station_lines.csv (18 rows, junction table)
- typecity_data/ridership.csv (3,024 rows)
- app.py — Streamlit choropleth dashboard